Comparative Analysis between BFS and DFS - Shortest Path Algorithms

Computer Science Department, College of Science and Humanities
Imam Abdulrahman Bin Faisal University, Jubail, Saudi Arabia
Supervised by: Dr. Azza A. Abdo

***Abstract*—Graph theory is a critical field in computer science and mathematics, particularly in solving shortest path problems. In this paper, a comparative analysis of Breadth-First Search (BFS) and Depth-First Search (DFS) algorithms for unweighted graphs, is presented. The algorithms efficiency analysis is measured, including execution time, and efficiency across varying graph structures such as level-order traversal (number of nodes through the path). The results indicate that BFS is more optimal than DFS for shortest path discovery due to its level-order traversal, despite its potential speed advantage, in practical BFS does not guarantee optimality. The insights derived from this study contribute to algorithm selection strategies in applications such as network routing, robotics, and artificial intelligence.**

Keywords— Graph Theory, Shortest Path Algorithms, Breadth-First Search (BFS), Depth-First Search (DFS), Algorithm Efficiency.

I. INTRODUCTION

Graph Graph theory is a foundational field in computer science and mathematics, dedicated to studying the relationships and interactions between discrete objects represented as vertices (nodes) and edges (links). One of the most critical challenges in this area is the shortest path problem, which involves determining the most efficient route between two nodes. This problem is pivotal not only from a theoretical standpoint but also due to its practical applications in transportation networks, telecommunications, and social network analysis [1][2].

Efficient solutions for the shortest path problem are essential for optimizing resource usage and enhancing performance in various real-world systems. Graph-based data structures are integral in modeling complex networks—ranging from social media platforms and urban transit systems to internet routing protocols. The quest for optimal paths in these systems has led to the development of numerous algorithms, each employing distinct traversal strategies.

Among the most prominent methods are Breadth-First Search (BFS) and Depth-First Search (DFS). BFS systematically explores the graph level by level, ensuring that the first occurrence of any node is along the shortest path. Conversely, DFS employs a deep traversal approach, which, while useful for exhaustive exploration, does not guarantee the shortest path and can result in increased computational overhead [3][4]. Moreover, the limitations of DFS in pathfinding have been discussed in depth in the literature, such as in the work by Baswana et al. [5].

In this paper, we present a comparative analysis of BFS and DFS algorithms for solving the shortest path problem in unweighted graphs. Our study evaluates these algorithms based on execution time, the number of nodes traversed, and overall efficiency across varying graph structures. By integrating empirical results with established theoretical insights, we aim to provide a comprehensive guide for selecting the most appropriate algorithm for specific application requirements.

The remainder of the paper is structured as follows: Section II reviews the background of BFS and DFS algorithms, Section III details the experimental setup and performance analysis, and Section IV concludes with a summary of our findings and suggestions for future research.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

II. BACKGROUND

A. *Breadth-First Search (BFS)*

BFS is a traversal algorithm that systematically explores all neighbors of a node before moving to the next level. This guarantees that when a node is encountered for the first time, it is via the shortest path in an unweighted graph. BFS operates using a queue data structure, where nodes are enqueued in the order they are discovered and dequeued when processed. This ensures an optimal traversal pattern suitable for applications such as network routing, AI pathfinding, and web crawling.

BFS explores nodes level by level using a queue structure, ensuring that the first time a node is reached, it is via the shortest path.

Time Complexity: O (V + E), where V is the number of vertices and E is the number of edges.

Space Complexity: O(V), due to the queue storage.

The steps which illustrate the BFS traversal algorithm is shown in algorithm.1.

Algorithm.1: BFS.

```
function bfs_shortest_path(graph, start, target):
    initialize queue with start node
    initialize visited set
    initialize parent dictionary to reconstruct the path

    while queue is not empty:
      current_node = dequeue from queue
      if current_node is target:
        return reconstruct_path(parent, start, target)

    if current_node not in visited:
        add current_node to visited
        for each neighbor in graph[current_node]:
          if neighbor is not in visited:
            enqueue neighbor to queue
             set parent[neighbor] = current_node
    return "No path found"
```

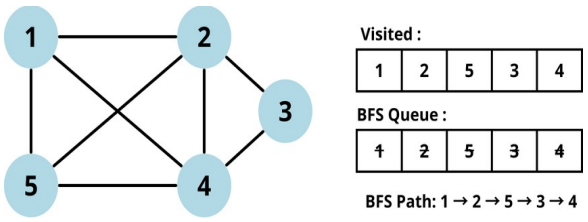


Fig.1 An example of a graph used for BFS traversal.

Fig.1 illustrates an example of a graph used for BFS traversal. The BFS algorithm utilizes a queue to keep track of nodes to explore next, ensuring that nodes are visited in the order they were discovered.

B. *Depth First Search (DFS)*

DFS follows a different strategy by exploring as far as possible along a branch before backtracking. The first approach utilizes a stack (either explicitly or via recursion) to track visited nodes. While DFS can efficiently explore large graphs, it does not guarantee finding the shortest path. It is particularly useful in scenarios requiring exhaustive exploration, such as maze solving, scheduling, and game tree analysis[5].

DFS explores paths deeply before backtracking. While DFS finds a path, it does not guarantee the shortest path without an exhaustive search.

Time Complexity: O (V + E).

Space Complexity: O(V), particularly in recursive implementations.

The steps which illustrate the DFS traversal algorithm is shown in algorithm.2.

Algorithm.2: DFS.

```
function dfs_shortest_path(graph, current_node,
target, visited, path):
    add current_node to visited
    append current_node to path

    if current_node is target:
       return path

    for each neighbor in graph[current_node]:
       if neighbor is not in visited:
          result = dfs_shortest_path(graph, neighbor,
target, visited, path)
          if result is not None:
             return result
    path.pop()  # backtrack
    return None
```

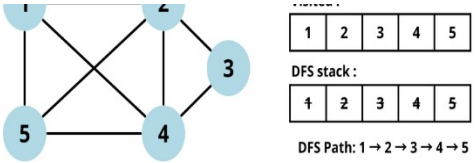


Fig.2 An example of a graph used for DFS traversal.

This diagram in Fig.2 illustrates the Depth-First Search (DFS) traversal of the given graph, showcasing the order in which nodes are visited and the final DFS path from the starting node. The DFS stack maintains the nodes to be explored, following a Last In, First Out (LIFO) order, which allows the algorithm to backtrack effectively.

C. Shortest Path Finding in Unweighted Graphs`

To solve this problem, BFS is often employed due to its level-order exploration, which guarantees that the shortest path is discovered first. The algorithm maintains a queue for node traversal and a parent dictionary to reconstruct the path once the destination node is reached. By backtracking from the destination to the starting node, the shortest path can be efficiently reconstructed.

While DFS can also be utilized for pathfinding, it is generally less efficient for shortest path discovery in unweighted graphs. DFS explores nodes deeply before backtracking, which may lead to longer paths being discovered before the shortest one. To ensure the shortest path is found using DFS, all possible paths must be explored and compared, increasing computational cost and execution time. Consequently, BFS is preferred for the shortest path finding in unweighted graphs.

This problem has significant theoretical and practical implications, impacting domains such as navigation systems, network routing, and resource allocation. Understanding and implementing efficient shortest path algorithms is essential for optimizing performance in these applications.

III. RESULTS AND DISCUSSION

A. Code Implementation

BFS and DFS algorithms are implemented using Python programming code. The experiments were conducted on an HP Victus Gaming Laptop, equipped with Windows 11 Home and powered by an 11th Gen Intel Core i5-12500H processor with 16 GB of RAM. The program generates a random undirected graph represented as an adjacency list, with vertices indexed by integers for straightforward access and manipulation. A generate_random_graph() function creates edges while avoiding self-loops and duplicate connections.

The provided code defines two key functions: bfs_shortest_path and dfs_shortest_path . BFS guarantees finding the shortest path due to its level-order traversal, while DFS explores paths in a depth-first manner, which may result in longer routes being discovered first.

Both algorithms operate on unweighted graphs, treating all edges as equal in weight, allowing them to efficiently identify paths based solely on connectivity. The measure_performance function assesses the execution time and memory usage of each algorithm, providing insights into their performance characteristics.

Experiments were conducted on randomly generated unweighted graphs with varying densities and connectivity levels, highlighting the differences in runtime efficiency between BFS and DFS for the shortest path finding problem. To find the shortest paths between vertices, bfs_shortest_path employs BFS to explore vertices level by level using a queue while maintaining a record of visited nodes for path reconstruction, operating with a time complexity of O(V + E). Conversely, dfs_shortest_path utilizes DFS to explore paths deeply before backtracking, also achieving a time complexity of O(V + E), but without guaranteeing the discovery of the shortest path. The generated test graph is determined by user-defined variables for vertices and edges. Summary for each function and its computational complexities are summarized in table.1 The efficiency of each function is illustrated in Table 1.

The implemented code generates a random undirected graph represented as an adjacency list, with vertices indexed by integers. It computes the shortest paths between vertices using Breadth-First Search (BFS) and Depth-First Search (DFS).

As discussed in Table 1, generate_random_graph function constructs edges while preventing self-loops and duplicates. This is achieved by generating random integer pairs for edges and ensuring that no vertex connects to itself (self-loop) and that no duplicate edges are formed between any two vertices. Specifically, while generating edges, a check is performed to confirm that the vertex pairs are distinct and not already included in the adjacency list.

BFS traverses vertices in a level-order fashion utilizing a queue. This queue enables the algorithm to explore all neighbors of a vertex before moving on to the next level of vertices. The visited nodes are meticulously tracked to facilitate path reconstruction, achieving a time complexity of O(V+E) by calculate_vertices_and_edges function.

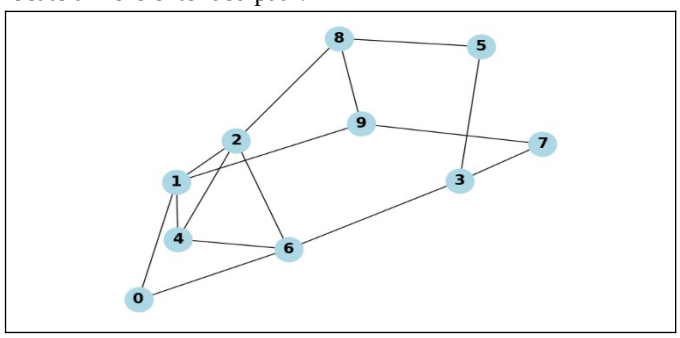
In contrast, DFS employs a stack (or the call stack via recursion) to explore paths more deeply before backtracking. This method allows the algorithm to navigate through a vertex's neighbors, pushing each unvisited neighbor onto the stack until it reaches a dead end. Like BFS, DFS also has a time complexity of O(V+E) by calculate_vertices_and_edges function, but it does not guarantee the shortest path.

Table.1 Function Descriptions and Time Complexity Analysis.

Function Name	Description	Time Complexity
generate_random_graph(num_vertices, num_edges)	Generates a random undirected graph with a specified number of vertices and edges. Returns an adjacency list representation of the graph.	O(E) Complexity arises from generating unique edges, with time varying based on the number of edges relative to vertices.
draw_graph(graph)	Prints the adjacency list representation of the graph.	O(V) Iterates through all vertices to print connections, resulting in linear complexity relative to the number of vertices.
bfs_shortest_path(graph, start, end)	Performs a Breadth-First Search (BFS) to find the shortest path between two nodes in an unweighted graph.	O(V + E) Every vertex and edge is visited at most once, resulting in linear complexity.
dfs_shortest_path(graph, start, end)	Performs a Depth-First Search (DFS) to find a path between two nodes in an unweighted graph.	O(V + E) Similar to BFS, DFS explores each vertex and edge, leading to linear time complexity.
measure_performance(search_algorithm, graph, start, end)	Measures the time taken and memory usage of a given search algorithm (BFS or DFS) on the graph.	O(V + E) Complexity is determined by the search algorithm (BFS or DFS) called within the function.
calculate_vertices_and_edges(graph)	Calculates the number of vertices (V) and edges (E) in the graph.	O(V) Iterates through the adjacency list to count vertices and edges, visiting each vertex once.

The program evaluates execution time and number of nodes in the path between two node for both algorithms and generates a test graph represented as an adjacency list using draw_graph(graph).

Fig.3.a shows an example for testing the program with two algorithms BFS and DFS. When applying BFS on the graph for Fig.3.a to get the path from node 2 to 7, and the path was [2,8,9,7] Time Taken was 0.000026s. While after applying DFS, the path was [2,8,5,3,6,0,1,9,7], and time Taken: 0.000195s. The memory space was the same for two algorithms, it was118.19 MB as shown in Fig.3.b. In the example for Fig.3, BFS was able to find the shorter path faster than DFS, which took longer to locate a more extended path.



(a)

```
======== BFS and DFS Algorithms ========

BFS: Shortest path from node 2 to node 7:
 [2, 8, 9, 7]
 Time Taken: 0.000026s
 Memory Usage: 148.05 MB
 Time Complexity: O(V + E)
 Time Complexity: O(10 + 15)

DFS: Shortest path from node 2 to node 7:
 [2, 8, 5, 3, 6, 0, 1, 9, 7]
 Time Taken: 0.000195s
 Memory Usage: 148.05 MB
 Time Complexity: O(V + E)
 Time Complexity: O(10 + 15)
```

(b)

Fig.3: (a) A graph with 10 vertices and 15 edges, and (b) applying BFS and DFS algorithms on the graph in (a).

B. *Comparison of Algorithms*

The equations are an exception to the prescribed specifications of this template. You will need to determine whether or not your equation should be typed using either the Times New Roman or the Symbol font (please no other font). To create multileveled equations, it may be necessary to treat the equation as a graphic and insert it into the text after your paper is styled.

Experiments were conducted with some samples of graphs, each differing in the number of vertices and edges, demonstrating the algorithms' effectiveness and efficiency across various scenarios. The characteristics for graph list sample are shown in Table.2. A comparative analysis of the BFS and DFS algorithms in the numerical graph reveals key differences in their performance.

Table.2: List of graph sample characteristics.

List of graph samples	Graph	Graph	Graph	Graph	Graph	Graph	Graph	Graph	Graph	Graph
Graph Nodes	10	50	100	200	300	500	600	700	800	1000
Graph Edges	20	100	200	1000	3000	5000	700	8000	500	8000

The comparative analysis of the BFS and DFS algorithms reveals significant performance differences when applied to 10 graphs which characterized by varying numbers of vertices and edges, along with targeted nodes. Table.3 summerized the path founded after applying BFS and DFS algorithms for the graph sample list. It is shown from results in table.3 that the path found when applyidng BFS algorithm is shorter than the path

aftre applying DFS algorithm. The comparative analysis of the Breadth-First Search (BFS) and Depth-First Search (DFS) algorithms BFS is able to find the shorter path faster than DFS, which took longer to locate a more extended path.

Also, in Fig.4, it shows a comparison between BFS vs. DFS Performance running time based on the graph sample list. The results indicate that DFS has the optimal practical time compared to BFS in running time. On the other hand, BFS the results indicate that BFS is more optimal than DFS for shortest path discovery due to its level-order traversal. Fig.5 illustrates the number of traversed nodes between BFS and DFS based on the graph sample list.

Table.3: Graphs sample tested related to nodes and edges

List of graph sample	Target path		Number of traversed nodes	
	Start Node	End Nodes	BFS	DFS
Graph_1	4	5	3	4
Graph_2	44	1	5	11
Graph_3	37	67	4	12
Graph_4	199	70	4	12
Graph_5	90	140	4	7
Graph_6	348	37	4	56
Graph_7	123	426	8	21
Graph_8	114	662	3	25
Graph_9	179	510	4	33
Graph_10	672	781	3	102

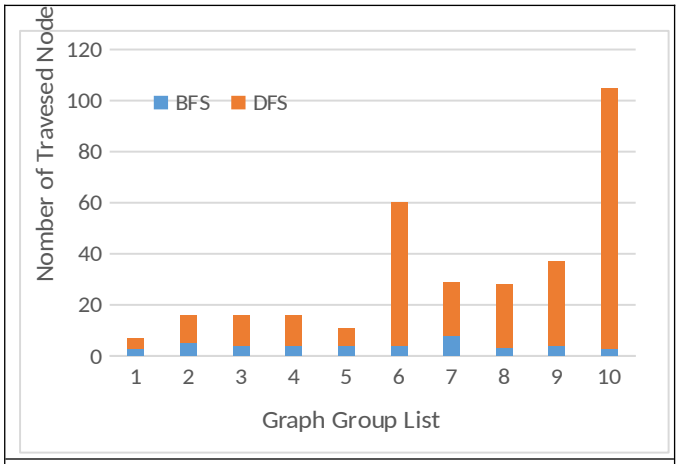


Fig.5: Number of traversed nodes between BFS and DFS based on the graph sample list.

IV. Conclusion

The comprehensive experimental evaluation conducted in this study demonstrates distinct performance trade-offs between the Breadth-First Search (BFS) and Depth-First Search (DFS) algorithms when applied to the shortest path problem in unweighted graphs. Our analysis, grounded in both theoretical complexity (as detailed in Table.1) and empirical performance (illustrated in Tables.2 and 3, as well as Figures 3, 4, and 5), confirms that BFS reliably identifies the shortest path through its level-order traversal strategy.

Specifically, the results indicate that BFS consistently traverses fewer nodes and requires less time to construct the shortest path, as evidenced by the comparative measurements in Table.3 and the timing results shown in Fig.3. Conversely, while DFS occasionally exhibits competitive runtime under certain conditions (as suggested by the data in Fig.4), its inherent deep traversal strategy generally results in longer, suboptimal paths, thereby limiting its practical applicability in scenarios where path optimality is paramount.

The implementation of these algorithms using Python, along with the generation of random graphs with varied characteristics (Table.2), provides further empirical validation of the theoretical time and space complexities. Overall, our findings substantiate that algorithm selection should be informed by the specific requirements of the application domain: BFS is preferable when optimal path length is critical, whereas DFS may be advantageous in contexts where rapid execution is prioritized over path optimality.

In conclusion, our rigorous comparative analysis of BFS and DFS for solving the shortest path problem clearly demonstrates that BFS outperforms DFS in terms of optimality and computational efficiency. These findings, corroborated by both theoretical and experimental evidence, reinforce the notion that algorithm selection must be tailored to the unique demands of the application—favoring BFS for optimal routing in network communication and urban transportation, and considering DFS when exhaustive search is required despite the potential for increased path lengths.

Acknowledgment

We would like to extend our sincere gratitude to everyone who contributed to the successful completion of this research. We are especially grateful to our supervisors for their invaluable guidance, insightful feedback, and continuous support throughout the research process. Additionally, we would like to express our appreciation to the members of the Department of Computer Science at Imam Abdulrahman Bin Faisal University for their assistance and encouragement.

References

[1] "A Survey of Shortest-Path Algorithms," arXiv, May 2017.

[2] "An Extensive Review of Shortest Path Problem Solving Algorithms," IEEE Xplore, 2021.

[3] "Comparison Study of Three Shortest Path Algorithms," IEEE Xplore, 2019.

[4] "Analysis and Performance Evaluation of Various Shortest Path Algorithms," IEEE Xplore, 2023.

[5] S. Baswana, A. Goel, and S. Khan, "Incremental DFS algorithms: A theoretical and experimental study," Proceedings of the Twenty-Ninth Annual ACM-SIAM Symposium on Discrete Algorithms, pp. 53–72, Society for Industrial and Applied Mathematics, 2018.